《猫语心声》全栈开发路线图 v2

项目定位： 2D 单机治愈系猫咖模拟经营游戏 | **引擎：** Unity 6 + TEngine 框架 | **美术风格：** 2D 手绘暖调治愈风（参考锈湖/纸嫁衣） | **场景切换：** 移步换景式 | **开发周期：** 约 16 周（4 个月 Demo 版）

目录

《猫语心声》全栈开发路线图 v2 1
一、项目总览与前置修改说明 2
1.1 相对于 v1 版本的三大核心修改 2
1.2 架构迁移对照：Godot → Unity + TEngine 2
1.3 TEngine 框架核心模块速览 2
1.4 游戏核心系统一览 3
1.5 移步换景式场景架构说明 3
二、全局里程碑与阶段划分 4
2.1 16 周 4 阶段总览 4
2.2 各阶段各职位主责 4
三、策划路线图 5
3.1 P0 — 原型验证阶段（W1-W4） 5
四、引擎路线图 6
4.1 P0 — 原型验证阶段（W1-W4） 6
4.2 P1 — 核心搭建阶段（W5-W8） 9
4.3 P2 — 深度内容阶段（W9-W12） 12
4.4 P3 — 打磨发布阶段（W13-W16） 12
五、程序路线图 13
5.1 P0 — 原型验证阶段（W1-W4） 13
5.2 P1 — 核心搭建阶段（W5-W8） 15
5.3 P2 — 深度内容阶段（W9-W12） 17
5.4 P3 — 打磨发布阶段（W13-W16） 18
六、美术路线图 20
6.1 P0 — 原型验证阶段（W1-W4） 20
6.2 P1 — 核心搭建阶段（W5-W8） 21
6.3 P2 — 深度内容阶段（W9-W12） 22
6.4 P3 — 打磨发布阶段（W13-W16）（同 v1，风格适配） 23
七、角色协作依赖矩阵 24
7.1 关键交付物依赖链 24
7.2 每周协作要点 24
八、技术风险与缓解 25
8.1 单机版 AI 性能风险（同 v1 + 新增 Unity 特有问题） 25
8.2 2D 手绘暗调风格风险（新增） 25
8.3 进度/资源风险（同 v1 + Unity 特有） 25
8.4 Unity+TEngine 特有风险（新增） 25
附录：关键参考文档索引 27

一、项目总览与前置修改说明

1.1 相对于 v1 版本的三大核心修改

修改项	v1 版本	v2 版本
游戏引擎	Godot 4 (GDScript)	**Unity 6 + TEngine 框架 (C# / HybridCLR 热更)**
美术风格	2D 像素风	**2D 手绘明亮调治愈风**
场景架构	等距/TileMap 连续场景	**移步换景式房间制**（固定机位，点击切换房间，每间房为独立手绘场景）

1.2 架构迁移对照：Godot → Unity + TEngine

Godot 原组件	Unity + TEngine 替代方案
GDScript	C# (.NET 9.0)，热更代码置于 GameScripts/HotFix/GameLogic/
Autoload 单例	GameModule.XXX 静态访问（Timer/Scene/Audio/UI/Fsm/Resource 等）
Godot Signal	GameEvent (int/接口事件) + EventInterface Source Generator
.tscn 场景文件	Unity Scene (.unity) + TEngine SceneModule.LoadSceneAsync()
.tres 资源定义	Luban Excel 配置表 → .bytes → ConfigSystem.Instance.Tables
GDScript Resource	ScriptableObject + YooAsset 资源管理 (LoadAssetAsync<T>)
AnimatedSprite2D	Unity Animator + SpriteRenderer (2D Sprite Animation)
TileMap	手绘全屏场景图 + 点击热区(Clickable Area)
ParallaxBackground	多层 Sprite 前景/背景 parallax + 后处理 Volume
CanvasLayer UI	TEngine UIWindow/UIWidget + UGUI Canvas
AnimationTree	Unity Animator Controller + Blend Tree
AudioStreamPlayer	TEngine AudioModule (AudioType.Music/Sound/UISound)
Godot Shader	Unity Shader Graph / HLSL + URP RendererFeature
GDExtension(C++)	无需（C# Job System / Burst Compiler 替代性能关键路径）

1.3 TEngine 框架核心模块速览

TEngine 框架核心模块

模块/组件	描述
GameModule.Timer	计时器系统（AddTimer / RemoveTimer）
GameModule.Scene	场景管理（LoadSceneAsync / UnloadAsync）
GameModule.Audio	音频管理（Music / Sound / UISound）
GameModule.UI	UI 管理（ShowUIAsync / CloseUI / HideUI）
GameModule.Fsm	有限状态机（CreateFsm / ChangeState）
GameModule.Resource	YooAsset 资源加载（LoadAssetAsync 等）
GameEvent	全局事件系统（Send / AddEventListener）
ConfigSystem	Luban 配置表访问（Tables.TbXxx）
MemoryPool	内存池（Acquire / Release）

1.4 游戏核心系统一览

层级	包含模块/功能
模拟经营层	猫咖房间布局 · 经济双轨 · 昼夜循环 · 道具库存 · 顾客 NPC · 成就系统
AI 认知层	猫语心声 · 意图决策（MDP+Transformer） · 记忆管理 · 性格过滤器 · 心声 LLM 生成
情感疗愈层	流浪猫救助 · 信任度阶段 · 深度互动 · 猫咪社交涌现 · 夜话叙事
叙事驱动层	常规任务 · 动态任务 · 里程碑事件 · 记忆相册 · 信任称号 · 剧情时间线

1.5 移步换景式场景架构说明

移步换景是本作的核心场景呈现方式，参考锈湖系列与纸嫁衣系列的房间制探索：

- **固定机位**：每个房间/区域使用一个精心构图的固定视角，如同观赏一幅手绘插画
- **点击切换**：玩家点击场景中的门、走廊、楼梯等交互热点，触发场景切换
- **过渡效果**：房间切换使用淡入淡出/推拉/溶解等过渡动画（可配置）
- **场景层级**：每个房间场景包含 背景层 → 家具层 → 角色层 → 光照后处理层
- **独立氛围**：不同房间有独立的色调、光影和 BGM 参数

猫咖房间拓扑（示例）：

```
                    ┌──────────────┐
                    │ 阳光温室     │
                    │(后院玻璃房)  │
                    └──────────────┘
                          │ 推门
┌──────────────┐  ┌──────────────┐  ┌──────────────┐
│ 静音隔间     │──│ 主大厅       │──│ 后院花园     │
│(隔离疗愈)    │  │(核心经营)    │  │(室外区域)    │
└──────────────┘  └──────────────┘  └──────────────┘
                          │ 楼梯
                    ┌──────────────┐
                    │ 二楼阁楼     │
                    │(玩家住所)    │
                    └──────────────┘
```

二、全局里程碑与阶段划分

2.1 16 周 4 阶段总览

阶段	周次	里程碑	交付标准
P0: 原型验证	W1-W4	核心循环可玩	主大厅单场景 + 1 只猫完整交互闭环 + 移步换景切换 + 基础 UI
P1: 核心搭建	W5-W8	三大系统上线	3 只原住猫完整 AI + 经营经济循环 + 多房间切换 + 昼夜系统
P2: 深度内容	W9-W12	长线玩法成型	流浪猫救助全流程 + 动态任务 + 社交涌现 + 全房间场景
P3: 打磨发布	W13-W16	可演示 Demo	性能达标 + 手绘美术完整 + 音效适配 + Bug 修复

2.2 各阶段各职位主责

阶段	策划	引擎	程序	美术
P0	系统详细规则定稿	Unity+TEngine 项目搭建/URP 管线	核心交互原型	风格指南/猫设原画/主大厅概念
P1	数值填表/任务脚本	场景管理/动画系统/Shader	AI 系统/经营系统	全房间场景/UI 组件/猫咪动画
P2	流浪猫剧本/AI 提示词	性能优化/资源管线	救助系统/社交系统	流浪猫立绘/后处理特效
P3	数值调优/验收测试	打包发布配置	Bug 修复/存档完善	音效/动画润色/宣传素材

三、策划路线图

> 策划路线图的核心内容（系统规则、数值设计、AI 行为规则、任务设计、流浪猫剧本）与 v1 版本保持一致，以下仅标注因引擎和美术风格变更而调整的关键项。

3.1 P0 — 原型验证阶段（W1-W4）

W1: 系统规则详细定稿

- [ ] **猫咖房间系统规则定稿（新增）**
  - 定义房间拓扑结构：主大厅、静音隔间、后院花园、阳光温室、二楼阁楼
  - 每个房间的解锁条件与环境属性映射（舒适度/刺激度/健康度基准值）
  - 房间面积与可放置家具数量上限
  - 移步换景的触发区域定义（门、走廊、楼梯的热区范围）
- [ ] **猫咖系统规则定稿**（同 v1）
  - 明确所有家具模板的`舒适度/刺激度/健康度`基础值计算公式
  - 定义环境属性对猫咪状态的影响映射表
  - 区域解锁条件（对应新房间拓扑）
- [ ] **经济系统数值初版**（同 v1）
- [ ] **三只原住猫详细设定定稿**（同 v1）

W2-W4（同 v1，略）

> 交互流程、道具系统、昼夜循环、AI 行为规则、原型验收等策划内容与 v1 基本一致。主要差异在于交互方式从"点击 TileMap 格子"变为"点击场景中的交互热点"。

四、引擎路线图

> 引擎岗位负责 Unity 项目工程架构、URP 渲染管线、TEngine 框架集成、场景管理、资源管线、Shader 开发、动画系统、音频系统、打包配置等底层基础设施。

4.1 P0 — 原型验证阶段（W1-W4）

W1: Unity + TEngine 项目搭建

- [ ] **项目初始化**
  - Unity 6 LTS 项目创建，目录结构规划：

```
CatVoice/
├── Configs/GameConfig/          # Luban 配置表工程
│   ├── Datas/                   # Excel 数据源
│   ├── gen_code_bin_to_project.bat
│   └── CustomTemplate/
└── UnityProject/Assets/
    ├── AssetRaw/                # 热更资源目录（YooAsset）
    │   ├── Actor/               # 角色 Prefab（猫咪、NPC）
    │   ├── Audios/BGM/ SFX/     # 音频资源
    │   ├── Configs/bytes/       # Luban 生成.bytes
    │   ├── Effects/             # 粒子特效、后处理资源
    │   ├── Scenes/              # 房间场景(.unity)
    │   │   ├── MainHall/        # 主大厅
    │   │   ├── QuietRoom/       # 静音隔间
    │   │   ├── Backyard/        # 后院花园
    │   │   ├── Greenhouse/      # 阳光温室
    │   │   └── Attic/           # 二楼阁楼
    │   ├── Shaders/             # Shader Graph / HLSL
    │   ├── Sprites/             # 手绘精灵图
    │   │   ├── Characters/      # 角色立绘、精灵
    │   │   ├── Scenes/          # 场景背景图
    │   │   ├── Furnitures/      # 家具精灵
    │   │   ├── UI/              # UI 精灵
    │   │   └── Effects/         # 特效精灵
    │   ├── UI/Atlas/ Prefabs/   # UI 资源
    │   └── Materials/           # 材质资源
    ├── Launcher/                # 启动器模块
    │   └── Scripts/             # LauncherMgr、加载 UI 等
    ├── TEngine/                 # TEngine 框架核心
    │   ├── Editor/              # 编辑器工具
    │   └── Runtime/             # 运行时核心
    └── GameScripts/             # 游戏逻辑脚本
        ├── GameEntry.cs         # 游戏启动入口
        ├── Procedure/           # 主包流程状态机
        └── HotFix/              # 热更代码
            ├── GameProto/       # Luban 生成代码
            └── GameLogic/       # 业务逻辑主开发区域
                ├── GameApp.cs   # 热更入口
                ├── System/      # 系统逻辑
                ├── Entity/      # 实体脚本（猫/玩家/NPC）
                ├── AI/          # AI 相关脚本
                ├── UI/          # UIWindow/UIWidget
                └── Event/       # EventInterface 定义
```

- [ ] **URP 渲染管线配置**
  - 创建 URP Asset（UniversalRenderPipelineAsset）
  - 2D Renderer 配置：启用 2D Sprite Lit（支持 2D 光照）
  - 目标分辨率：1920×1080，支持 16:9 窗口缩放
  - 抗锯齿：FXAA（2D 手绘风格无需 MSAA）

    - 后处理：启用 Volume 系统（Color Adjustments / Vignette / Bloom）
    - 手绘风格色彩空间：建议 Linear（更准确的光照混合）
- [ ] **TEngine 框架初始化**
    - 导入 TEngine UnityPackage
    - 配置 HybridCLR（热更程序集：GameProto、GameLogic）
    - 配置 YooAsset 资源收集器（AssetRaw 目录）
    - 验证启动流程：`ProcedureLaunch → ProcedureSplash → ... → ProcedureStartGame`
    - 确认 `GameApp.Entrance()` 热更入口正常调用
- [ ] **Editor Settings 配置**
    - 启用 C# strict null checking
    - 配置代码风格 `.editorconfig`
    - 设置 VS/Rider 外部编辑器集成

W2: 核心框架搭建

- [ ] **GameModule 核心模块初始化**
    - 确认模块启动顺序与依赖关系
    - 热更入口初始化流程：

```
// GameApp.cs — 热更入口
public static void Entrance(object[] objects)
{
    GameEventHelper.Init();                  // 1. 必须最先调用
    var hotfixAssemblies = objects[0] as List<Assembly>; // 2. 热更程序集列表
    Utility.Unity.AddDestroyListener(Release); // 3. 注册销毁回调
    StartGameLogic();                        // 4. 启动游戏逻辑
}
```

- [ ] **EventInterface 事件接口定义**

```
// IGameEvent.cs — 游戏全局事件接口
[EventInterface(EEventGroup.GroupUI)]
public interface ICatVoiceEvent
{
    void OnDayPhaseChanged(string newPhase);
    void OnCatInteracted(string catId, string action);
    void OnCurrencyChanged(string type, int newAmount);
    void OnQuestCompleted(string questId);
    void OnTrustMilestoneReached(string catId, string newStage);
    void OnItemUsed(string itemId, string targetId);
    void OnFurniturePlaced(string furnitureId, string roomId);
    void OnRoomChanged(string fromRoomId, string toRoomId);
}
```

> Source Generator 自动生成 `ICatVoiceEvent_Event` (int 常量) 和 `ICatVoiceEvent_Gen` (实现类)

- [ ] **移步换景场景系统设计**
    - 场景加载规范：
    - 主大厅场景始终驻留（Additive 加载其他房间或使用 Single 切换）
    - 推荐方案：每个房间为独立.unity 场景，使用 `LoadSceneMode.Single` + 过渡动画
    - `RoomTransitionData` ScriptableObject：定义房间连接关系图

```
// RoomTransitionData.cs
[CreateAssetMenu(menuName = "CatVoice/Room Transition Data")]
public class RoomTransitionData : ScriptableObject
{
    public string RoomId;
    public string RoomDisplayName;
    public string SceneLocation;        // YooAsset location
    public List<RoomExit> Exits;        // 出口列表
```

```
    public Color AmbientTint = Color.white;
    public string DefaultBGM;
}

[System.Serializable]
public class RoomExit
{
    public string TargetRoomId;
    public Vector2 HotspotPosition;      // 点击热区中心(归一化坐标 0-1)
    public Vector2 HotspotSize;
    public string ExitDescription;       // "通往后院"
}
```

W3: 2D 渲染与 Shader

- [ ] **手绘风格 Shader 库**
  - 猫耳视界后处理 Shader（URP Fullscreen Shader Graph）：
  - 色调偏移至暖黄
  - 边缘柔焦/水彩晕染效果
  - `_VisionStrength` 参数控制切换强度（0=正常，1=里世界）
  - 昼夜时间光照 Shader（2D Sprite Lit，通过 Global Light 2D 控制）：
  - 白天：暖黄环境光（色温 ~4000K）
  - 夜晚：暗蓝+暖灯点光（室内灯光的温暖对比）
  - 手绘纸张纹理叠加 Shader：
  - 全屏纸张纹理叠加（低不透明度）
  - 轻微的不规则边缘暗角（Vignette）
- [ ] **2D 光照系统**
  - URP 2D Light 使用：
  - Global Light 2D：全局环境光
  - Sprite Light 2D：窗户光斑、灯具
  - Point Light 2D：猫咪信任度光环
  - 阴影生成：Shadow Caster 2D（家具对地面的投影）
- [ ] **粒子与特效系统**
  - 治愈光芒粒子（Unity Particle System + Sprite Renderer）
  - 猫咪心情气泡（半透明 Image + DOTween/自写动画）
  - 信任度突破时光效（粒子爆发 + 屏幕边缘柔光）

W4: 原型集成支持

- [ ] **动画系统框架**
  - 猫咪 Sprite Animation 创建（基于 Sprite 切换的帧动画）
  - `Animator Controller` 结构设计：
  - Base Layer：基础移动状态（Idle / Walk / Run / Sleep）
  - Action Layer：交互动作（Eat / Play / Hide / Hiss / Stretch）
  - Emotion Layer：表情叠加（开心/恐惧/安心/好奇）
  - 意图枚举 → Animator Trigger 参数的映射配置表
- [ ] **音频系统基础**
  - AudioModule 配置：BGM/SFX/UISound 分类管理
  - BGM 播放/切换/淡入淡出（`GameModule.Audio.Play(AudioType.Music, ...)`）
  - SFX 播放接口封装：`AudioManager.Instance.PlaySfx(string location)`
  - 移步换景切换时 BGM 交叉淡入淡出
- [ ] **输入系统封装**
  - Unity Input System Package
  - 鼠标点击检测（场景交互热区 + UI 点击）

    - 键盘快捷键：空格键（猫耳视界开关）、ESC（暂停/设置菜单）
    - 移动端触控预留（单指点击/长按）

4.2 P1 — 核心搭建阶段（W5-W8）

W5: 资源管线

- [ ] **资源导入规范**
    - Sprite 导入设置：
    - 手绘精灵：Texture Type = Sprite (2D and UI)，Max Size = 2048 或 4096（全屏场景图）
    - 角色精灵：Max Size = 1024（单只猫立绘）
    - 压缩格式：ASTC (移动端) / DXT5 (PC)
    - Filter Mode：Bilinear（手绘风格，非像素风）
    - 音频导入：OGG Vorbis，BGM 流式(Streaming)，SFX 加载到内存(Decompress On Load)
    - 场景加载方式：通过 YooAsset 的 Scene location 加载
- [ ] **Luban 配置表体系搭建**
    - 创建 Excel 配置表：

表名	用途	导出模式
TbCat	猫咪基础数据（性格向量、品种、立绘 location）	map
TbFurniture	家具模板（舒适度/刺激度/价格/材质）	map
TbItem	道具模板（消耗品/交互品/环境改造品）	map
TbRoom	房间配置（场景 location、环境基准值）	map
TbIntent	意图枚举数据（触发条件、动画 trigger）	map
TbQuest	任务模板	map
TbDialogue	对话/心声文本库	map
TbEconomy	经济数值曲线参数	list

- 运行 `gen_code_bin_to_project.bat` 生成 C#代码和.bytes 数据
- 创建配置访问 Manager：`CatConfigMgr`、`FurnitureConfigMgr`、`ItemConfigMgr` 等
- [ ] **ScriptableObject 资产模板**

```
// CatDataSO.cs
[CreateAssetMenu(menuName = "CatVoice/Cat Data")]
public class CatDataSO : ScriptableObject
{
    public string CatId;
    public string DisplayName;
    public string Breed;
    [TextArea] public string Appearance;
    public float[] PersonalityVector = new float[8]; // 8 维性格向量
    [TextArea] public string Backstory;
    public CatType Type; // Native / Rescued
    public string SpriteLocation;    // YooAsset location
    public string PortraitLocation;
}
```

W6: 移步换景场景系统完善

- [ ] **房间场景管理器**

```
// RoomManager.cs (GameLogic/System/)
public class RoomManager
{
    private string _currentRoomId;
    private RoomTransitionData _currentRoomData;
    private readonly GameEventMgr _eventMgr = new();

    public async UniTask TransitionToRoom(string targetRoomId)
    {
        var fromRoom = _currentRoomId;
        GameEvent.Get<ICatVoiceEvent>().OnRoomChanged(fromRoom, targetRoomId);

        // 1. 播放过渡动画（淡出）
        await TransitionOut();

        // 2. 卸载当前场景 + 加载目标场景
        if (!string.IsNullOrEmpty(fromRoom))
            await GameModule.Scene.UnloadAsync(fromRoom);
        var scene = await GameModule.Scene.LoadSceneAsync(targetRoomId);

        // 3. 激活场景 + 过渡动画（淡入）
        GameModule.Scene.ActivateScene(targetRoomId);
        await TransitionIn();

        _currentRoomId = targetRoomId;
        // 4. 资源整理
        GameModule.Resource.UnloadUnusedAssets();
    }
}
```

- [ ] **交互热点系统（Clickable Area）**

```
// ClickableArea.cs — 挂载到场景中的交互热区 GameObject
public class ClickableArea : MonoBehaviour
{
    public enum AreaType { DoorExit, Furniture, Cat, Item, NPC, Decoration }
    public AreaType Type;
    public string TargetRoomId;         // DoorExit 时生效
    public string InteractionData;      // 传递给交互系统的参数

    private void OnMouseDown()
    {
        switch (Type)
        {
            case AreaType.DoorExit:
                GameEvent.Get<ICatVoiceEvent>().OnRoomChanged(CurrentRoom, TargetRoomId);
                break;
            case AreaType.Cat:
                // 触发猫咪交互菜单
                break;
            // ...
        }
    }
}
```

- [ ] **场景层级结构规范**

每个房间场景的标准 GameObject 层级：

```
Room_MainHall (Scene Root)
├── Background (SpriteRenderer — 房间背景手绘全图)
│   ├── FarLayer (远景装饰)
│   └── PaperTexture (纸张纹理叠加)
├── InteractionLayer
│   ├── Hotspots (ClickableArea 容器)
│   └── Furniture (已放置的家具实例)
├── CharacterLayer
```

```
│   ├── Cats (猫实例容器)
│   └── NPCs (顾客 NPC 容器)
├── Foreground (前景遮挡)
├── Lighting
│   ├── GlobalLight2D
│   ├── PointLights (灯具)
│   └── SpotLights (窗户光)
└── PostProcessVolume (房间独立后处理)
```

W7: UI 框架

- [ ] **UI 主题系统**
  - UGUI + 手账本视觉风格
  - 字体选择：
  - 中文正文：手写感圆体（如"方正喵呜体"风格）
  - 数字/英文：衬线字体（匹配手绘氛围）
  - 标题：毛笔手写体
  - 色板定义（暗调治愈系）：
  - 背景基调：暖深棕/暗米色（模拟旧纸张）
  - 强调色：琥珀橙 / 烛光金
  - 文字色：深褐（正文）/ 暖白（深色底）
  - 猫耳视界：蓝紫空灵色调
- [ ] **核心 UIWindow**

```
// 主界面 HUD
[Window(UILayer.UI, "HUDUI")]
public class HUDUI : UIWindow
{
    private int _fishCoin;
    private int _healLight;

    protected override void RegisterEvent()
    {
        AddUIEvent<string, int>(ICatVoiceEvent_Event.OnCurrencyChanged, OnCurrencyChanged);
        AddUIEvent<string>(ICatVoiceEvent_Event.OnDayPhaseChanged, OnDayPhaseChanged);
    }

    protected override void OnRefresh() { /* 刷新货币/时间/体力显示 */ }
}

// 猫语心声档案室（手账本）
[Window(UILayer.UI, "CatArchiveUI")]
public class CatArchiveUI : UIWindow { }

// 环形指令菜单（点击猫咪后弹出）
[Window(UILayer.Top, "CatInteractionMenuUI")]
public class CatInteractionMenuUI : UIWindow { }

// 场景过渡遮罩
[Window(UILayer.System, "RoomTransitionMaskUI")]
public class RoomTransitionMaskUI : UIWindow { }
```

- [ ] **UIWidget 子组件**
  - `CatEmotionWidget`：猫咪情绪雷达图 Widget
  - `ItemSlotWidget`：道具/背包格子 Widget
  - `FurnitureThumbWidget`：家具缩略图 Widget
  - `DialogueBubbleWidget`：心声气泡 Widget
- [ ] **环形指令菜单**
  - 点击猫咪 → 检测点击位置 → 以该位置为圆心展开环形菜单

    - 选项动态显隐：抚摸/喂食/玩耍/呼唤/轻声安抚（根据猫的状态动态调整）
    - 动画：DOTween 实现扇区旋转展开 + 选项缩放弹出

W8: 性能基准与优化

- [ ] **2D 渲染性能标准**
    - Draw Call：目标 < 150 per frame（URP 2D 批处理）
    - Sprite Atlas：按房间/功能分组打图集
    - 粒子数量上限：同屏最大 200 粒子
    - 后处理开销：优先使用 URP 内置 Volume 效果，避免自定义 Blit
- [ ] **内存预算**
    - 纹理内存：< 300MB（手绘高分辨率场景图占用较大）
    - 场景图按需加载：当前房间+相邻房间预加载，远距离房间卸载
    - 音频内存：BGM 流式播放，SFX < 32MB
    - 存档数据：单个存档 < 5MB

4.3 P2 — 深度内容阶段（W9-W12）

- [ ] **后处理特效系统完善**
    - 信任度突破特效：屏幕边缘 Bloom+暖光扩散
    - 猫耳视界过渡：URP Fullscreen Pass RendererFeature
    - 季节/天气视觉效果（可选）：窗外场景变化
- [ ] **性能剖析与优化**
    - Unity Profiler 使用：CPU/GPU 帧时间分析
    - AI 推理性能隔离（C# Task/Burst，不阻塞主线程）
    - LLM 文本生成异步化（UniTask + Loading 动画）
- [ ] **Mod 支持预留**（可选）
    - 自定义猫咪外观通过 ScriptableObject 扩展
    - 家具模板 Luban 配置表导入

4.4 P3 — 打磨发布阶段（W13-W16）

- [ ] **多平台打包配置**
    - Windows Standalone (Mono/IL2CPP)
    - macOS Standalone
    - 图标/启动画面/版本信息
    - HybridCLR 热更包构建流程
- [ ] **设置系统完善**
    - 分辨率/全屏/音量/语言切换
    - 存档管理（新建/删除/云备份预留）
- [ ] **崩溃日志与上报**
    - Unity Cloud Diagnostics / 自定义日志上报

五、程序路线图

> 程序岗位负责所有游戏逻辑系统的 C#实现，包括 AI 情绪引擎、猫行为系统、经营经济系统、任务系统、存档系统、UI 逻辑等。所有业务代码位于 `GameScripts/HotFix/GameLogic/` 中，遵循 TEngine 异步规范。

5.1 P0 — 原型验证阶段（W1-W4）

W1: 项目脚手架

- [ ] **与引擎协作搭建项目结构**
  - 确认目录结构/命名规范（参见引擎路线图 W1）
  - Git 仓库初始化 + `.gitignore`（排除 Library/Logs/Temp/导出文件/）
- [ ] **核心数据结构实现**

```
// CatState.cs
public class CatState
{
    // 8 维性格向量 [傲娇, 怯懦, 好奇, 贪吃, 社交, 攻击性, 活跃, 独立]
    public float[] Personality = new float[8];
    // 5 维情绪向量 [饥饿, 恐惧, 好奇, 舒适, 社交需求]
    public float[] Emotion = new float[5];
    public float TrustLevel;          // 0-100
    public float Energy;              // 0-1
    public float Health;              // 0-1
    public float Stress;              // 0-100
    public string CurrentRoomId;
    public float RelationshipToPlayer;
}

// MemoryItem.cs
public class MemoryItem : IMemory
{
    public string Desc;
    public float Timestamp;
    public float Importance;          // 0-10
    public float[] Embedding;         // 语义向量
    public string EventType;

    public void Clear() { Desc = null; Embedding = null; }
}
```

W2: 基础交互闭环（移步换景适配版）

- [ ] **鼠标交互系统**
  - 点击猫咪检测：`ClickableArea` 组件 + `OnMouseDown()`
  - 环形菜单弹出/隐藏（`GameModule.UI.ShowUIAsync<CatInteractionMenuUI>()`）
  - 选中猫咪高亮效果（SpriteRenderer.material 属性动画）
  - 点击门/通道 → 触发射线检测 → 发起房间切换
- [ ] **猫耳视界切换系统**

```
// CatVisionSystem.cs
public class CatVisionSystem
{
    private float _visionStrength; // 0=表世界, 1=里世界
    private bool _isHoldingSpace;

    public void OnSpaceKeyDown()
    {
        _isHoldingSpace = true;
       // 开始向 1.0 插值 _visionStrength
    }
```

```
    public void OnSpaceKeyUp()
    {
        _isHoldingSpace = false;
        // 开始向 0.0 插值 _visionStrength
    }

    private void UpdateVisionMaterial(float strength)
    {
        // 通过 Shader.SetGlobalFloat("_CatVisionStrength", strength)
        // 控制 URP Fullscreen Pass 的混合强度
        Shader.SetGlobalFloat("_CatVisionStrength", strength);
    }
}
```

- [ ] **玩家动作处理**
    - 动作类型枚举：

```
public enum PlayerAction { Pet, Feed, Call, Play, Ignore, Approach, Leave, Treat, Heal, Comfort }
```

- 动作→猫咪反应映射表（P0 阶段硬编码规则，后续接入 AI）

W3: 猫咪基础行为系统

- [ ] **意图-动画状态机**

```
// CatAgent.cs — 猫咪实体核心类
public class CatAgent
{
    public CatState State;
    public Animator Animator;
    public CatDataSO ConfigData;

    // 意图枚举（第一阶段 15 个）
    public enum Intent
    {
        IdleWander, FriendlyApproach, FearfulRetreat,
        PlayfulPounce, DemandFood, SocialGroom,
        CuriousInspect, Hide, Sleep, ScratchFurniture,
        AcceptPetting, HissWarning, InvestigateSound,
        FollowPlayer, StareAtWindow
    }

    public Intent CurrentIntent;

    public void ExecuteIntent(Intent intent)
    {
        CurrentIntent = intent;
        Animator.SetTrigger($"Intent_{intent}");
    }
}
```

- [ ] **猫咪 AI 基础规则引擎（W3-W4 硬编码版）**
    - 分层规则决策树（C#实现）：

```
读取当前状态(情绪/信任/环境) → 基于规则树计算意图分数 → 选取最高分意图
→ ExecuteIntent → 更新情绪值 → 反馈(动画+心声文本)
```

- [ ] **喵心声文本库**
    - 通过 Luban 配置表 `TbDialogue` 管理：意图×情绪→变体文本
    - 用于原型阶段演示（后续 P1 阶段替换为 LLM 生成）

W4: 原型集成（同 v1 流程，Unity 版）

5.2 P1 — 核心搭建阶段（W5-W8）

W5: AI 意图决策引擎

- [ ] **状态向量构建器**

```
// StateBuilder.cs
public class StateBuilder
{
    const int EmbedDim = 64;
    const int StateDim = 256;

    public float[] BuildStateVector(CatAgent cat)
    {
        // 拼接：性格(8) + 情绪(5) + 信任(1) + 精力(1) + 健康(1)
        //       + 环境特征(5) + 关系向量(5) + 玩家行为编码(10)
        // 各子特征经 Embedding 后拼接为 256 维
        // ...
    }
}
```

- [ ] **意图决策规则引擎**（W5-W6，先于 Transformer）
    - 完整版规则决策树（覆盖所有意图×所有情绪组合）
    - 决策频率：基于 `GameModule.Timer`，每 1 秒触发一次（不阻塞渲染）
    - 决策结果通过 `Log.Debug` 记录日志（用于收集训练数据）
- [ ] **性格过滤器实现**

```
// PersonalityFilter.cs
public class PersonalityFilter
{
    // 性格-意图兼容矩阵
    private Dictionary<string, Dictionary<string, float>> _compatibilityMatrix = new()
    {
        ["怯懦"] = new() { ["FriendlyApproach"] = -5f, ["Hide"] = 3f, ["CuriousInspect"] = -2f },
        ["好奇"] = new() { ["CuriousInspect"] = 3f, ["InvestigateSound"] = 2.5f },
        ["傲娇"] = new() { ["AcceptPetting"] = -2f, ["FollowPlayer"] = -1f },
    };

    public Dictionary<string, float> ApplyFilter(Dictionary<string, float> intentScores, float[] personality)
    {
        // 对每个意图的分数加上性格偏置
    }
}
```

W6: 记忆系统

- [ ] **工作记忆（固定窗口）**

```
// WorkingMemory.cs
public class WorkingMemory
{
    const int MaxSize = 20;
    private Queue<MemoryItem> _buffer = new();

    public void Add(MemoryItem item)
    {
        _buffer.Enqueue(item);
        if (_buffer.Count > MaxSize) _buffer.Dequeue();
    }
}
```

- [ ] **长期记忆（优先级队列+语义检索）**

```
// LongTermMemory.cs
public class LongTermMemory
{
    const int MaxCap = 500;
    private List<MemoryItem> _items = new();

    public void Add(MemoryItem item)
    {
        if (item.Importance < 4f) return;
        _items.Add(item);
        _items = _items.OrderByDescending(i => i.Importance).Take(MaxCap).ToList();
    }

    public List<MemoryItem> QueryBySimilarity(float[] queryEmbedding, int topK = 3)
    {
        // 余弦相似度检索
    }
}
```

W7: LLM 集成与文本生成

- [ ] **本地 LLM 方案评估**
    - 选项 A：llama.cpp + Unity 原生插件
    - 选项 B：ONNX Runtime + 量化模型
    - 选项 C：云端 API（OpenAI/Anthropic 等）
    - 推荐：**选项 C 为主（Demo 阶段效果优先）+ 选项 B 为本地降级**
- [ ] **LLM 服务封装**

```
// LLMService.cs
public class LLMService
{
    public async UniTask<string> GenerateInnerMonologue(
        string catName, string personalityDesc,
        Dictionary<string, float> emotion, string intentLabel,
        List<string> memoryContext, string playerAction)
    {
        var prompt = BuildPrompt(catName, personalityDesc, emotion, intentLabel, memoryContext, playerAction);
        try
        {
            return await CallCloudLLM(prompt);
        }
        catch (Exception ex)
        {
            Log.Warning($"LLM call failed, fallback to text library: {ex.Message}");
            return GetFallbackText(intentLabel, emotion); // 降级到 Luban 文本库
        }
    }
}
```

- [ ] **备用文本库**（Luban `TbDialogue` 表，意图×情绪→文本）

W8: 经营系统实现

- [ ] **经济系统**

```
// EconomyManager.cs
public class EconomyManager
{
    public int FishCoin { get; private set; }
    public int HealLight { get; private set; }

    public void EarnFishCoin(int amount, string source)
    {
```

```
        FishCoin += amount;
        GameEvent.Get<ICatVoiceEvent>().OnCurrencyChanged("fish_coin", FishCoin);
    }

    public bool SpendFishCoin(int amount, string purpose)
    {
        if (FishCoin < amount) return false;
        FishCoin -= amount;
        GameEvent.Get<ICatVoiceEvent>().OnCurrencyChanged("fish_coin", FishCoin);
        return true;
    }
    // HealLight 同理
}
```

- [ ] **猫咖环境数值系统**
  - 遍历当前房间内所有已放置家具，累加 comfort/stimulation/hygiene
  - 家具耐久度衰减（基于 `GameModule.Timer` 每小时触发）
  - 环境数值→猫咪状态影响映射
- [ ] **昼夜循环系统**

```
// DayCycleManager.cs
public class DayCycleManager
{
    public enum Phase { Preparation, Daytime, Night }
    public Phase CurrentPhase;
    public float GameTime;
    public int DayCount;
    public string Weather;

    public async UniTaskVoid AdvanceTime(float delta)
    {
        GameTime += delta;
        // 检查阶段切换 + 触发事件
    }
}
```

- [ ] **道具与库存系统**
  - 消耗品叠加/使用/耗尽
  - 交互道具耐久度管理
  - 家具购买/放置/移动/维修（与移步换景房间绑定）

5.3 P2 — 深度内容阶段（W9-W12）

W9-W10: 流浪猫救助系统

- [ ] **救助进度追踪**

```
// RescueProgress.cs
public class RescueProgress
{
    public enum Stage { Alienated, Exploring, Tentative, Trusting, Healed }
    public Stage CurrentStage;
    public float TrustLevel; // 0-100

    public void UpdateTrust(float delta)
    {
        var oldStage = CurrentStage;
        TrustLevel = Mathf.Clamp(TrustLevel + delta, 0f, 100f);
        CheckStageTransition(oldStage); // 触发里程碑事件/称号
    }
}
```

- [ ] **专属疗愈方案系统**（同 v1 逻辑）
- [ ] **领养系统**（同 v1 逻辑）

W10-W11: 猫咪社交涌现系统

- [ ] **社交模拟引擎**

```
// SocialSimulation.cs
public class SocialSimulation
{
    public async UniTaskVoid SimulateNightSocial()
    {
        var cats = GetAllCats();
        var pairs = SelectInteractionPairs(cats);
        foreach (var pair in pairs)
        {
            var interaction = GenerateInteraction(pair.CatA, pair.CatB);
            RecordInteraction(interaction);
            CheckDynamicQuestTrigger(interaction);
        }
    }
}
```

- [ ] **猫际关系网络**（亲密度-100~+100，同 v1 逻辑）
- [ ] **夜话对话生成**（LLM 生成 + 次日提示）

W11-W12: 动态任务与成就系统（同 v1 逻辑）

- [ ] **任务系统**：任务模板→实例化→状态管理→条件检测
- [ ] **成就系统**：记忆相册/信任里程碑/拍照功能

W12: Transformer 策略网络集成

- [ ] **模型导入与推理**：ONNX Runtime Unity 插件
- [ ] **混合策略切换**：规则引擎为主 → Transformer 预测 → LLM 文本

5.4 P3 — 打磨发布阶段（W13-W16）

W13: 存档系统完善

- [ ] **本地存档结构**

```
// SaveData.cs
[System.Serializable]
public class SaveData
{
    public string SaveName;
    public int FishCoin;
    public int HealLight;
    public float GameTime;
    public string DayPhase;
    public string Weather;
    public string CurrentRoomId;     // 新增：玩家所在房间
    public List<CatSaveData> Cats;
    public List<FurnitureSaveData> Furnitures;
    public List<InventorySaveData> Inventory;
    public List<QuestSaveData> Quests;
    public List<AchievementSaveData> Achievements;
    public List<PhotoSaveData> Photos;
    public Dictionary<string, object> CustomerProgress;
}
```

- [ ] **存档加密/校验**（SHA256 + 版本号迁移）

W14-W16: 同 v1（Bug 修复、边缘情况、性能优化、最终集成测试）

六、美术路线图

> 美术岗位负责所有视觉与听觉资产的创作。**v2 最大变化：美术风格从 2D 像素风转向 2D 手绘暖色调治愈风，场景架构改为移步换景房间制。**

6.1 P0 — 原型验证阶段（W1-W4）

W1: 美术风格指南

- [ ] **视觉风格定调（核心变更）**
  - **参考作品：**
  - **锈湖系列(Rusty Lake)：**沉静的手绘质感、暗调室内场景、略微超现实的氛围、固定镜头构图
  - **纸嫁衣系列：**东方民俗手绘风格、暗调中的温暖色调点缀、人物与场景的同风格融合
  - **本作风格定位：**"暗调手绘治愈风" ——
  - 基调：沉静的暗色背景（深棕/暗灰绿），营造夜晚猫咖的私密感
  - 灵魂：暖黄灯光/烛光的局部照亮，形成冷暖对比
  - 质感：手工笔触纹理、旧纸张质感、轻微做旧感
  - 情绪：安静但不阴森，治愈但不甜腻
  - **色板定义：**
  - 背景基调：暖深棕 `#3A2C24` / 暗灰绿 `#2D3A2C` / 墨蓝 `#1C2833`
  - 灯光强调：琥珀暖黄 `#E8B86D` / 烛光橙 `#D4956B`
  - 点缀色：铜绿 `#6B8E7B` / 褪色玫瑰 `#C49B9B`
  - 猫耳视界：蓝紫空灵 `#5B6EAD` → `#3A2870`
  - UI 纸张：米白`#F5ECD7` / 浅棕 `#D9C8A9`
  - **角色比例：**3-4 头身（偏写实 Q 版融合，类似锈湖角色比例但更圆润）
- [ ] **设计语言规范**
  - 线条风格：有笔触质感的不规则手绘线条（非均匀粗细）
  - 光影处理：暗调环境 + 局部暖光聚焦（类似伦勃朗光的戏剧感）
  - 纹理叠加：纸张纹理 + 轻微噪点 + 边缘做旧
  - 整体氛围关键词：安静、私密、温暖角落、猫咪的庇护所

W2: 三只原住猫角色设计

- [ ] **奥利奥（奶牛猫）**
  - 立绘：体态浑圆，黑白花色，右耳尖缺口，沉稳的坐姿
  - 表情集（手绘线稿+上色）：高冷闭眼 / 暗中观察（半睁眼）/ 偷乐（嘴角微翘）/ 炸毛 / 安心眯眼
  - 关键帧动画草稿（Sprite 序列帧）：待机呼吸 / 走动 / 蹭腿 / 睡觉
  - 设计重点：虽为黑白配色，但黑色部分用暖深棕而非纯黑，融入整体色调
- [ ] **小雪（纯白波斯猫）**
  - 立绘：异色瞳（蓝+琥珀），毛发蓬松略有打结感，身体微蜷的姿态
  - 表情集：惊恐 / 小心翼翼 / 好奇偷看 / 安心眯眼 / 分离焦虑
  - 关键帧动画草稿：缩成一团 / 侧头观察 / 缓慢靠近 / 蹭手 / 躲藏
  - 设计重点：白色毛发在暗调场景中作为"高光"存在，异色瞳为视觉焦点
- [ ] **橘子（橘猫幼猫）**
  - 立绘：圆滚滚幼猫体型，天真无邪大眼睛
  - 表情集：贪吃渴望 / 好奇瞪眼 / 得意 / 困倦 / 开心
  - 关键帧动画草稿：蹦跳 / 扑玩具 / 偷吃 / 打滚 / 睡觉
  - 设计重点：橘色为暗调场景中最温暖的色彩元素

W3: 猫咖场景概念设计（移步换景制）

- [ ] **主大厅概念图（核心场景）**
  - 固定机位：略微俯视的 45°视角（不是完全的正交等距）
  - 画面构图：
  - 前景：吧台一角 / 木质桌面
  - 中景：猫咪活动区（猫爬架/软垫/食盆）
  - 远景：大窗户（可看到夜色街景）/ 通往其他房间的门
  - 光源：天花板暖黄吊灯 + 窗户透入的月光
  - 氛围：老旧猫咖但正在被悉心经营，安静而温暖
- [ ] **房间出口/入口视觉规范**
  - 每个可切换区域的门/通道需要有视觉引导（微弱的光从门缝透出、门框轻微泛光）
  - 门/楼梯需有清晰的热区视觉暗示（但不过分突兀）
- [ ] **基础家具概念（手绘线稿）**
  - 猫爬架(2-3 种，木质+剑麻材质)
  - 猫窝垫(2-3 种，毛毡/棉布质感)
  - 互动玩具(逗猫棒/毛线球)
  - 猫食盆/水盆（陶瓷质感）

W4: UI 概念设计

- [ ] **UI 风格指南**
  - 核心概念：**"夜话手账本"** —— 一本旧皮革封面的笔记本，记录猫咖的日常
  - 面板风格：做旧纸张背景 + 手写体文字 + 印章/标签贴纸装饰
  - 按钮风格：纸质标签 / 火漆印章
  - 字体选择（同引擎 UI 部分）
- [ ] **核心 UI 线稿**
  - HUD 布局：柔和不突兀的底部/顶部信息条
  - 环形指令菜单：围绕点击位置展开的古铜色环形菜单
  - 手账本(图鉴)界面：翻页动画，每只猫一页
  - 猫耳视界切换：屏幕边缘泛出蓝紫光晕，中心保留暖色

6.2 P1 — 核心搭建阶段（W5-W8）

W5-W6: 场景美术量产

- [ ] **猫咖主大厅完整手绘场景**
  - 全幅场景原画（1920×1080 基准，可等比放大至 4K）
  - 图层分离：背景 → 中景家具 → 前景 → 光照叠加层（供引擎分层使用）
  - 昼夜变体：白天版（窗外日光+室内暖调）、夜晚版（窗外暗蓝+室内暖灯对比强烈）
  - 出口标记：门、通往静音隔间、后院花园的视觉元素
- [ ] **扩展区域概念图**
  - 静音隔间：更小的房间、昏暗温暖灯光、柔软织物质感
  - 后院花园：室外夜景、月光+串灯照明、植物
  - 阳光温室：玻璃屋顶透下月光、植物剪影、温暖湿润感
  - 二楼阁楼：斜屋顶、天窗、私密空间感

W6-W7: 猫咪动画量产

- [ ] **三只原住猫完整动画集**
  - 帧动画（12fps，手绘序列帧）：

    - 每个意图对应至少一个动画（约 15 个意图×3 只猫）
    - 关键动画：Idle / Walk / Sleep / Eat / Play / Hide / Hiss / Stretch / Groom
    - 每只猫约需 8-12 帧/动画（比像素风更多，手绘风格需要更流畅的中间帧）
- [ ] **情绪可视化系统**
    - 心情气泡颜色编码（与猫耳视界统一的色系）
    - 信任度光环：Sprite Light 2D，随信任度变化亮度和范围

W7-W8: UI 资产量产

- [ ] **完整 UI 素材包**（手账本风格）
    - 面板背景：做旧纸张纹理（多页变体）
    - 按钮四态：Normal/Hover/Pressed/Disabled（纸质标签风格）
    - 图标：小鱼币(铜币质感) / 治愈光芒(暖光水晶质感) / 家具/道具图标
    - 猫咪头像小图标（手绘圆框头像）
- [ ] **手账本界面**
    - 封面/内页装饰/标签页图标（皮革封面+铜扣）
    - 猫咪立绘展示区（内页插画风格）
    - 情绪雷达图组件（手绘墨迹风格）
- [ ] **字体集成**（同引擎 UI 部分）

W8: 音效初版（同 v1）

- 环境音、猫咪音效、交互音效（暗调氛围需更注重环境音的空间感和沉浸感）

6.3 P2 — 深度内容阶段（W9-W12）

W9-W10: 流浪猫美术

- [ ] **首批流浪猫(≥3 只)角色设计**
    - 每只：立绘/表情集/动画集（手绘）
    - 创伤视觉体现：瘦弱的体态、蓬乱的毛发、警惕的眼神
    - 逐渐康复的视觉变化：毛发渐变光泽、体态变圆润、眼神变柔和
    - 多阶段立绘：每个信任阶段一张立绘变体

W10-W11: 特效

- [ ] **猫耳视界水彩后处理**
    - URP Fullscreen Shader Graph
    - 色调从暖暗调 → 蓝紫空灵色调的渐变
    - 边缘水彩晕染效果（Sobel 边缘检测+柔焦）
    - 心声气泡：半透明深底+白色手写体文字+轻微漂浮动画
- [ ] **情感特效**
    - 信任度突破：暖金色粒子从屏幕边缘涌入+Bloom
    - 治愈光芒获取：暖光从猫咪位置扩散
    - 成就解锁：手账本翻页+印章盖下动画

W11-W12: NPC 美术（同 v1 风格适配）

W12: BGM 制作

- 暗调室内氛围音乐（钢琴+弦乐+环境音）
- 白天 BGM：轻柔的吉他/钢琴，慵懒的咖啡店氛围
- 夜晚 BGM：更安静的钢琴独奏，略带感伤

- 猫耳视界 BGM：空灵的合成器+钟琴

6.4 P3 — 打磨发布阶段（W13-W16）（同 v1，风格适配）

七、角色协作依赖矩阵

7.1 关键交付物依赖链

```
策划 ──→ 引擎：系统规则 → 场景/UI 框架设计
策划 ──→ 程序：数值表/AI 规则/任务配置(Luban 表) → 系统实现
策划 ──→ 美术：猫设/房间描述/UI 需求（手绘暗调风） → 视觉资产创作
引擎 ──→ 程序：TEngine 框架/API/资源管线 → 系统接入
引擎 ──→ 美术：导入规范/Shader 接口/Animator 状态机 → 资产对接
美术 ──→ 程序：手绘精灵/Sprite Animation/UI 素材/音效 → 脚本绑定
程序 ──→ 策划：可运行版本 → 数值验证/体验测试
```

7.2 每周协作要点

周次	策划→程序	策划→美术	程序→美术	引擎→程序
W1	系统规则文档	美术风格需求（手绘暖色）	—	TEngine 项目结构/代码规范
W2	交互流程说明	猫设详细描述	猫 Sprite 规格要求	URP 管线/输入系统
W3	AI 规则矩阵	房间概念需求（移步换景）	Animator 状态机接口	GameEvent/GameModule 框架
W4	原型验收标准	UI 概念反馈	—	动画/音频/Shader 接口
W5	家具/道具数值表(Luban)	房间场景全幅原画	猫咪动画序列帧列表	资源管线/YooAsset 对接
W6	经济曲线数据	UI 素材清单	LLM 文本显示需求	移步换景场景拼接
W7	任务模板库	NPC 设计需求	对话/心声 UI 样式	UI 框架/UIWindow
W8	流浪猫设定	流浪猫美术需求	意图-动画 Trigger 映射表	场景管理完善
W9	流浪猫救助剧本	流浪猫全美术	救助进度 UI 需求	后处理 Shader
W10	社交涌现规则	NPC 全美术	LLM 对话生成接口	BGM 音轨对接
W11	动态任务触发条件	情感特效需求	任务/成就 UI 绑定	性能剖析
W12	数值平衡 V2	宣传素材需求	拍照功能对接	资源优化
W13-16	持续验收/调优	持续验收/补缺	持续 Bug 修复	打包/兼容性测试

八、技术风险与缓解

8.1 单机版 AI 性能风险（同 v1 + 新增 Unity 特有问题）

风险	影响	缓解措施
本地 LLM 推理延迟过大	心声文本生成等待时间过长	①异步生成+Loading 气泡动画(UniTask) ②优先使用云端 API(Demo 阶段) ③Luban 文本库作为降级
Transformer 策略网络训练数据不足	意图预测不准确	①先用规则引擎运行完整 Demo ②离线收集数据后训练 ③演示时使用规则+随机混合策略
多只猫(≥5 只)同时 AI 推理性能不足	帧率下降	①C# Task 分帧处理 ②意图决策降至 0.5Hz(Timer 驱动) ③Burst Compiler 加速数值计算

8.2 2D 手绘暗调风格风险（新增）

风险	影响	缓解措施
暗调氛围过于阴森，与"治愈"定位冲突	玩家感受偏离预期	①严格控制暖光点缀的比例(每场景至少 2-3 处暖光源) ②猫咪角色保持柔和圆润设计 ③UI 层使用温暖的纸张色调作对比
手绘场景资源量大(全屏原画)	内存/包体过大	①场景背景使用 2048 分辨率(支持 2K 显示) ②图层分离设计，复用通用元素 ③ASTC 压缩 ④按需加载(只驻留当前+相邻房间)
移步换景切换生硬	破坏沉浸感	①设计多种过渡动画(淡入淡出/推拉/溶解) ②过渡时短暂显示场景名称手写字 ③相邻房间预加载减少等待
手绘序列帧动画工作量大	每只猫 15 个动画×8-12 帧=120-180 帧	①先做核心动画(5 个)，再逐步扩充 ②利用 Unity Sprite Resolver/Sprite Library 管理变体 ③某些非关键动画可减少帧数

8.3 进度/资源风险（同 v1 + Unity 特有）

风险	影响	缓解措施
4 个月内 AI 系统无法达到预期	Demo 演示效果打折	①P0-P1 使用规则引擎+文本库 ②AI 作为增强层而非基础层
TEngine 框架学习成本	团队初期效率降低	①P0 阶段引擎岗位先搭建好模板 ②程序直接按模板写业务代码
HybridCLR 热更调试复杂	开发效率降低	①P0-P2 大部分时间关闭热更，直接运行 ②P3 阶段再启用热更验证

8.4 Unity+TEngine 特有风险（新增）

风险	影响	缓解措施
YooAsset 资源管理不熟悉	资源加载/释放问题	①严格遵循 `LoadAssetAsync` → `UnloadAsset`

		配对 ②Sprite 用 SetSprite 扩展(内置缓存池) ③GameObject 用 LoadGameObjectAsync
UniTask 异步编程规范不熟悉	异步 Bug	①禁止使用 Coroutine 和同步加载 ②CancellationToken 防止销毁后回调
Luban 配置表使用不熟练	数值/文本配置效率低	①W5 引擎建立完整模板后策划填入 ②配置表变更→一键生成→热更
URP 2D 光照与手绘场景的协调	光照效果不理想	①W1-W2 进行 URP 2D 光照预研验证 ②使用 2D Sprite Lit 材质+法线贴图(可选)增强立体感

附录：关键参考文档索引

文档名称	主要内容	用途
《猫语心声》策划案（草案）	游戏概述/系统介绍/角色设定/UI 设计	策划：设计依据
技术策划案	AI 技术方案(MDP+Transformer+LLM+性格过滤器+记忆管理)	程序：AI 系统实现参考
《猫语心声》全栈开发路线图 v1	v1 版本 Godot 方案	对比参考
TEngine 框架文档	GameModule API / UIWindow 生命周期 / 事件系统 / 资源管理 / Luban 配置	引擎+程序：框架使用参考
锈湖系列游戏美术分析	手绘暗调场景/固定镜头构图/氛围营造	美术：风格参考
纸嫁衣系列美术分析	东方手绘质感/暗调场景/房间制探索	美术：风格参考

> **文档版本:** v2.0 | **编制日期:** 2026-05-08 | **变更概要:** 引擎迁移至 Unity6+TEngine / 美术风格转为手绘暖调治愈风(锈湖+纸嫁衣参考) / 场景架构改为移步换景房间制 | **下次修订:** 根据 P0 原型验证反馈调整